Tugas Kelompok ke-3

Week 8

Algorithmic Analysis, Mathematical Induction, and Recursive Functions

Learning Outcome:

- LO2: Analyze algorithm complexity: Students will be able to assess the time and space complexity of algorithms

---

Instruksi Pengerjaan

1. Buat program menggunakan Java dengan nama file PlaylistRekursif.java.
2. Gunakan array objek Lagu (dari tugas sebelumnya) untuk menyimpan data playlist.
3. Implementasikan fungsi-fungsi berbasis rekursif untuk melakukan operasi analisis pada playlist.
4. Tulis komentar di setiap fungsi utama untuk menjelaskan:
   - Tujuan fungsi
   - Base case dan recursive case
   - Kompleksitas waktunya
5. Tambahkan fitur pengukuran waktu eksekusi (time measurement) yang ada dalam Java.
6. Sertakan identitas kelompok di bagian atas file (nama, NIM, kelas).
7. Kumpulkan hasil dalam bentuk:
   - File `.java`
   - Screenshot hasil eksekusi setiap operasi
   - Link GitHub/Replit/video demo
   - Laporan PDF singkat yang berisi analisis *time and growth rate* untuk setiap fungsi rekursif.
   - Pastikan semua berkas diunggah ke LMS.

Deskripsi Tugas

Kelompok diminta untuk mengembangkan sistem manajemen playlist musik yang telah dibuat pada tugas sebelumnya, dengan menambahkan kemampuan analisis rekursif dan pengukuran efisiensi algoritmik.

Program ini harus mampu:

1) Menghitung total durasi playlist menggunakan rekursi.
2) Menampilkan daftar lagu secara terbalik (reverse traversal) menggunakan rekursi.
3) Menentukan lagu dengan durasi terpanjang menggunakan rekursi.
4) Menganalisis kompleksitas waktu dan growth rate dari fungsi-fungsi tersebut.

Struktur Program yang Diharapkan

1. **Class Lagu**
   - Gunakan kembali dari Tugas Kelompok 1 (atribut: judul, artis, durasi).
2. **Class PlaylistRekursif**
   - Memiliki atribut Lagu[] playlist.
   - Memiliki fungsi-fungsi:
     - ✓ static double totalDurasi(Lagu[] list, int n);
     - ✓ static void tampilkanMundur(Lagu[] list, int index);
     - ✓ static double cariDurasiTerpanjang(Lagu[] list, int index);
   - Tambahkan pengukuran waktu eksekusi untuk masing-masing fungsi.

Soal:

1. Implementasi Fungsi Rekursif Dasar – (Bobot 40%)
   - `totalDurasi()` → menghitung total seluruh durasi lagu.
   - `tampilkanMundur()` → menampilkan lagu dari akhir ke awal.
   - `cariDurasiTerpanjang()` → mencari lagu dengan durasi terpanjang.

```
Total Durasi Lagu = 21.36 menit
Daftar Lagu (Terbalik):
Fix You - Coldplay (4.52)
Shivers - Ed Sheeran (3.50)
Perfect - Ed Sheeran (4.23)
Lagu Terpanjang: Fix You (4.52 menit)
```

2. Analisis Kompleksitas Waktu (Time and Growth Rate) – (Bobot 30%)

   Tuliskan analisis **Big-O** untuk setiap fungsi:

Fungsi	**Base Case**	**Growth Rate**	**Kompleksitas**
totalDurasi()			
tampilkanMundur()			
cariDurasiTerpanjang()			

   Sertakan grafik sederhana (dalam laporan) yang menggambarkan *growth rate* linear terhadap ukuran data playlist (misal, n = 2, 4, 6, 8, 10).

3. Eksperimen Pengukuran Waktu Eksekusi (Bobot 20%)
   - Ukur dan tampilkan **waktu eksekusi (dalam milidetik)** untuk setiap fungsi.
   - Jalankan program dengan jumlah data berbeda (misal: 3, 5, dan 10 lagu).
   - Bandingkan hasil waktu eksekusi dan buat tabel analisis di laporan:

Jumlah Lagu (n)	totalDurasi()	tampilkanMundur()	cariDurasiTerpanjang()
3	1 ms	1 ms	0 ms
5	2 ms	2 ms	1 ms
10	4 ms	4 ms	3 ms

4. Kerapian Program dan Dokumentasi (Bobot 10%)
   a. Kode tersusun rapi, menggunakan indentasi yang konsisten.
   b. Setiap fungsi utama memiliki komentar.
   c. Output program tampil jelas dan mudah dibaca.

Output yang diharapkan

```
=== ANALISIS REKURSIF PLAYLIST ===
Jumlah lagu : 5
Total durasi : 20.35 menit
Lagu terpanjang : "Viva La Vida" - Coldplay (4.52 menit)

Daftar lagu (ditampilkan terbalik):
5. Viva La Vida
4. Yellow
3. Fix You
2. Shivers
1. Perfect

Execution Time (totalDurasi): 3 ms
Execution Time (tampilkanMundur): 2 ms
Execution Time (cariDurasiTerpanjang): 3 ms
```

LEMBAR JAWABAN

Nama Kelompok: [Group - 7]
Anggota:

1. [JIMMY SEPRIANSYAH] - [2902706364]
2. [ANITA] - [2902734600]
3. [CHARYS SONNIA INDRIYANA] - [2902728251]
4. [SHAFA ALIFIA SALSABILA NURUL ICHSAN] - [2902697233]
5. [MUHAMMAD RIZKI KURNIA PUTRA] - [2401968362]

Kelas: [Computer Science]

1. Implementasi Fungsi

```
/**
 * totalDurasi() - REKURSIF
 * Tujuan: Menghitung total durasi seluruh lagu
 * Base case: n == 0 → return 0
 * Recursive case: durasi[n-1] + totalDurasi(n-1)
 * Kompleksitas waktu: O(n)
 */
static double totalDurasi(int n) {
    if (n == 0) return 0;
    return playlist[n-1].durasi + totalDurasi(n-1);
}

/**
 * tampilkanMundur() - REKURSIF
 * Tujuan: Menampilkan lagu dari indeks terakhir ke pertama
 * Base case: index < 0 → stop
 * Recursive case: tampilkan index, lalu panggil index-1
 * Kompleksitas waktu: O(n)
 */
static void tampilkanMundur(int index) {
    if (index < 0) return;
    System.out.println((index+1) + ". " + playlist[index].judul +
            " - " + playlist[index].artis +
            " (" + playlist[index].durasi + " menit)");
    tampilkanMundur(index - 1);
}

/**
 * cariDurasiTerpanjang() - REKURSIF
 * Tujuan: Mencari durasi terpanjang dalam playlist
 * Base case: n == 1 → return durasi[0]
 * Recursive case: max(durasi[n-1], cariDurasiTerpanjang(n-1))
 * Kompleksitas waktu: O(n)
 */
static double cariDurasiTerpanjang(int n) {
    if (n == 1) return playlist[0].durasi;
    double maxSebelumnya = cariDurasiTerpanjang(n - 1);
    return Math.max(playlist[n-1].durasi, maxSebelumnya);
}
```

2. Analisa KomplesitasWaktu

Fungsi	Base Case	Recursive Case	Growth Rate	Kompleksitas
totalDurasi()	n == 0 → return 0	durasi[n-1] + totalDurasi(n-1)	Linear	O(n)
tampilkanMundur()	index < 0 → return	cetak lalu panggil tampilkanMundur(index-1)	Linear	O(n)
cariDurasiTerpanjang()	n == 1 → return durasi[0]	max(durasi[n-1], cariDurasiTerpanjang(n-1))	Linear	O(n)

3. Ekperimen Pengukuran Waktu

Metode pengukuran waktu dilakukan menggunakan System.nanoTime() di Java, kemudian dikonversi ke milidetik (ms).

Dengan menggunakan 3 scenario jumlah lagu yang berbeda : 3 lagu, 5 lagu, 10 lagu

Jumlah Lagu	totalDurasi()	tampilkanMundur()	cariDurasiTerpanjang()
3	0ms	15ms	0ms
5	0ms	14ms	0ms
10	0ms	18ms	0ms

Catatan: angka diatas bisa bervariasi tergantung spesifikasi perangkat dan kondisi JVM

```
=== ANALISIS REKURSIF PLAYLIST ===

Jumlah lagu: 3
Total durasi: 9.85 menit
Execution Time (totalDurasi): 0 ms

Lagu terpanjang: "Somebody to Love" (4.55 menit)
Execution Time (cariDurasiTerpanjang): 0 ms

Daftar lagu (ditampilkan terbalik):
3. We Will Rock You - Queen (2.01 menit)
2. Don't Stop Me Now - Queen (3.29 menit)
1. Somebody to Love - Queen (4.55 menit)
Execution Time (tampilkanMundur): 15 ms
```

```
=== ANALISIS REKURSIF PLAYLIST ===

Jumlah lagu: 5
Total durasi: 17.25 menit
Execution Time (totalDurasi): 0 ms

Lagu terpanjang: "Somebody to Love" (4.55 menit)
Execution Time (cariDurasiTerpanjang): 0 ms

Daftar lagu (ditampilkan terbalik):
5. We Will Rock You - Queen (2.01 menit)
4. Don't Stop Me Now - Queen (3.29 menit)
3. Somebody to Love - Queen (4.55 menit)
2. Another One Bites the Dust - Queen (3.32 menit)
1. Under Pressure - Queen (4.08 menit)
Execution Time (tampilkanMundur): 14 ms
```

```
Jumlah lagu: 10
Total durasi: 38.54 menit
Execution Time (totalDurasi): 0 ms

Lagu terpanjang: "Bohemian Rhapsody" (5.55 menit)
Execution Time (cariDurasiTerpanjang): 0 ms

Daftar lagu (ditampilkan terbalik):
10. We Will Rock You - Queen (2.01 menit)
9. Don't Stop Me Now - Queen (3.29 menit)
8. Somebody to Love - Queen (4.55 menit)
7. Another One Bites the Dust - Queen (3.32 menit)
6. Under Pressure - Queen (4.08 menit)
5. Radio Ga Ga - Queen (5.48 menit)
4. I Want to Break Free - Queen (3.43 menit)
3. The Show Must Go On - Queen (4.24 menit)
2. We Are the Champions - Queen (2.59 menit)
1. Bohemian Rhapsody - Queen (5.55 menit)
Execution Time (tampilkanMundur): 18 ms
```

4. Dokumentasi

```java
/**
 * Tugas Kelompok ke-3 (Week 8)
 * Topik: Algorithmic Analysis, Mathematical Induction, and Recursive
Functions
 *
 * Nama Kelompok: [Group - 7 ]
 * Anggota:
 * 1. [JIMMY SEPRIANSYAH] - [2902706364]
 * 2. [ANITA] - [2902734600]
 * 3. [CHARYS SONNIA INDRIYANA] - [2902728251]
 * 4. [SHAFA ALIFIA SALSABILA NURUL ICHSAN] - [2902697233]
 * 5. [MUHAMMAD RIZKI KURNIA PUTRA] - [2401968362]
 * Kelas: [Computer Science]
 */

import java.util.Scanner;

class LaguRekursif {
    String judul, artis;
    double durasi;

    LaguRekursif(String judul, String artis, double durasi) {
        this.judul = judul;
        this.artis = artis;
        this.durasi = durasi;
    }
}

public class PlaylistRekursif {
    static LaguRekursif[] playlist = new LaguRekursif[50];
    static int jumlahLagu = 0;
    static Scanner sc = new Scanner(System.in);

    /**
     * totalDurasi() - REKURSIF
     * Tujuan: Menghitung total durasi seluruh lagu
     * Base case: n == 0 → return 0
     * Recursive case: durasi[n-1] + totalDurasi(n-1)
     * Kompleksitas waktu: O(n)
     */
    static double totalDurasi(int n) {
        if (n == 0) return 0;
        return playlist[n-1].durasi + totalDurasi(n-1);
    }

    /**
     * tampilkanMundur() - REKURSIF
     * Tujuan: Menampilkan lagu dari indeks terakhir ke pertama
     * Base case: index < 0 → stop
     * Recursive case: tampilkan index, lalu panggil index-1
     * Kompleksitas waktu: O(n)
     */
```

```java
    static void tampilkanMundur(int index) {
        if (index < 0) return;
        System.out.println((index+1) + ". " + playlist[index].judul +
                " - " + playlist[index].artis +
                " (" + playlist[index].durasi + " menit)");
        tampilkanMundur(index - 1);
    }

    /**
     * cariDurasiTerpanjang() - REKURSIF
     * Tujuan: Mencari durasi terpanjang dalam playlist
     * Base case: n == 1 → return durasi[0]
     * Recursive case: max(durasi[n-1], cariDurasiTerpanjang(n-1))
     * Kompleksitas waktu: O(n)
     */
    static double cariDurasiTerpanjang(int n) {
        if (n == 1) return playlist[0].durasi;
        double maxSebelumnya = cariDurasiTerpanjang(n - 1);
        return Math.max(playlist[n-1].durasi, maxSebelumnya);
    }

    /**
     * Mencari judul lagu dengan durasi terpanjang
     */
    static String cariJudulTerpanjang(int n, double durasiTerpanjang) {
        if (n == 0) return "";
        if (playlist[n-1].durasi == durasiTerpanjang) {
            return playlist[n-1].judul;
        }
        return cariJudulTerpanjang(n-1, durasiTerpanjang);
    }

    public static void main(String[] args) {
        // Data playlist untuk 10 lagu
        playlist[jumlahLagu++] = new LaguRekursif("Bohemian Rhapsody",
"Queen", 5.55);
        playlist[jumlahLagu++] = new LaguRekursif("We Are the Champions",
"Queen", 2.59);
        playlist[jumlahLagu++] = new LaguRekursif("The Show Must Go On",
"Queen", 4.24);
        playlist[jumlahLagu++] = new LaguRekursif("I Want to Break Free",
"Queen", 3.43);
        playlist[jumlahLagu++] = new LaguRekursif("Radio Ga Ga", "Queen",
5.48);
        playlist[jumlahLagu++] = new LaguRekursif("Under Pressure",
"Queen", 4.08);
        playlist[jumlahLagu++] = new LaguRekursif("Another One Bites the
Dust", "Queen", 3.32);
        playlist[jumlahLagu++] = new LaguRekursif("Somebody to Love",
"Queen", 4.55);
        playlist[jumlahLagu++] = new LaguRekursif("Don't Stop Me Now",
"Queen", 3.29);
        playlist[jumlahLagu++] = new LaguRekursif("We Will Rock You",
"Queen", 2.01);


        System.out.println("=== ANALISIS REKURSIF PLAYLIST ===\n");
```

```
        System.out.println("Jumlah lagu: " + jumlahLagu);

        // 1. Total Durasi
        long start = System.nanoTime();
        double total = totalDurasi(jumlahLagu);
        long end = System.nanoTime();
        System.out.printf("Total durasi: %.2f menit\n", total);
        System.out.println("Execution Time (totalDurasi): " + (end -
start) / 1_000_000 + " ms\n");

        // 2. Lagu Terpanjang
        start = System.nanoTime();
        double durasiTerpanjang = cariDurasiTerpanjang(jumlahLagu);
        String judulTerpanjang = cariJudulTerpanjang(jumlahLagu,
durasiTerpanjang);
        end = System.nanoTime();
        System.out.printf("Lagu terpanjang: \"%s\" (%.2f menit)\n",
judulTerpanjang, durasiTerpanjang);
        System.out.println("Execution Time (cariDurasiTerpanjang): " +
(end - start) / 1_000_000 + " ms\n");

        // 3. Tampilkan Mundur
        start = System.nanoTime();
        System.out.println("Daftar lagu (ditampilkan terbalik):");
        tampilkanMundur(jumlahLagu - 1);
        end = System.nanoTime();
        System.out.println("Execution Time (tampilkanMundur): " + (end -
start) / 1_000_000 + " ms\n");
    }
```

Link Repo

https://github.com/charys-si/R1_TK3-COSC6025-Data-Structures-and-Algorithm-Analysis